

C++&Qt Day3 hw2 보고서

휴머노이드 수습단원 이주원

목차

1. 프로젝트 개요
2. 파일 구조
3. 코드 구조
4. 실행결과
5. 어려웠던 점

1. 프로젝트 개요

이 프로젝트는 격자 맵 위에서 A*와 다익스트라를 동시에 돌려 비교하는 프로그램이다. 마우스로 벽, 시작점, 도착점을 찍고 시작 버튼을 누르면 왼쪽 맵에서는 에이스타가, 오른쪽 맵에서는 다익스트라가 각각 실행되어 경로와 방문한 칸이 색으로 표시되고, 아래 라벨에 경로 비용, 방문한 노드 수, 걸린 시간이 뜬다.

Day1, 2에서 배운 클래스 개념과 qt의 시그널, 슬롯 활용에 더해, 이번에는 QWidget을 직접 상속받아 커스텀 위젯을 만들고 paintEvent, mousePressEvent 같은 가상 함수를 오버라이드 하는 방법을 학습하는데에 초점을 두었다. 화면에 그리는 부분을 직접 구현해야 하기 때문에 위젯이 언제 다시 그려지는지, update()가 어떤 역할을 하는지를 이해하는 것이 중요했다.

또한 두 알고리즘의 코드가 거의 같고 우선순위 큐에 넣을 때 휴리스틱을 더하느냐 마느냐의 차이 하나뿐인데도 방문하는 칸의 개수가 크게 달라진다는 점을 직접 눈으로 확인하는 것에 의의를 두었다.

2.파일 조구

hw3

```
|—— hw3_report.pdf
|—— Project_name
|   |—— CMakeLists.txt
|   |—— include
|       |—— mainwindow.h      //클래스 설계도
|   |—— resources
|   |—— src
|       |—— main.cpp          //진입점
|       |—— mainwindow.cpp    //구현
|   |—— ui
|       |—— mainwindow.ui     //버튼등 화면 배치
```

3. 코드 구조

헤더파일

class map_widget : public QWidget 맵 한 장을 담당하는 커스텀 위젯이다.
public에는 set_mode, run_dijkstra, run_astar, set_wall, set_start, set_goal, get_visited, get_cost, set_size이 있다.
protected에는 paintEvent와 mousePressEvent가 있다. 큐티가 알아서 불러주는 함수라 직접 호출하지 않고 오버라이드만 해두면 된다.
private에는 맵 정보와 계산에 쓰이는 자료들이 있다. w, h는 가로 세로 칸 수이고 map은 0이면 벽, 1이면 길이다. g는 시작점부터의 실제 비용, parent는 어느 칸에서 왔는지, closed는 확정된 칸인지, path는 최종 경로이다. 맵은 2차원이지만 vector는 1차원으로 잡고 $i = y * w + x$ 로 인덱스를 계산하는 방식을 썼다. 따라서 만약 w로 나누면 나머지는 x 몫은 y라 풀어놓기도 쉽다.

Cpp

MainWindow::MainWindow(QWidget *parent) 생성자이다. 먼저 두 맵 위젯을 서로의 twin으로 연결해 준다.

cpp

```
ui->map_area->twin = ui->map_area2;    // 서로 짝 지어주기
```

```
ui->map_area2->twin = ui->map_area;
```

이렇게 해두면 한쪽 맵에 벽을 찍었을 때 반대쪽 맵에도 똑같이 찍히게 만들 수 있다. 두 알고리즘을 비교하려면 맵이 완전히 같아야 하기 때문에 필요한 부분이다.

그 다음 connect가 들어간다. wall, start, goal 버튼은 각각 set_mode(0,1,2)를 두 위젯 모두에 호출해서 클릭 모드를 바꾼다. run 버튼은 QElapsedTimer로 시간을 재면서 A*와 다익스트라를 차례로 실행하고, 결과를 라벨에 적는다. size apply 버튼은 스펀박스의 값을 읽어 맵 크기를 바꾼다.

map_widget::map_widget(QWidget *parent) 생성자이다. map을 전부 1(길)로 채워 초기화한다.

`void map_widget::set_size(int s)` 맵 크기를 바꾸는 함수이다. `w`, `h`를 `s`로 바꾸고 `map`을 다시 잡는다. 이때 칸 번호가 전부 무효가 되기 때문에 `start`와 `goal`을 `-1`로 리셋하고 나머지 `vector`들도 비워 준다.

`void map_widget::set_mode(int m)` 마우스 클릭이 벽을 찍는 건지 시작점을 찍는 건지 정하는 함수이다.

`void map_widget::set_wall(int i)` 해당 칸이 벽이면 길로, 길이면 벽으로 바꾼다. 토글 방식이라 잘못 찍었을 때 다시 눌러서 지울 수 있다.

`void map_widget::set_start(int i) / set_goal(int i)` 시작점과 도착점을 정한다. 그 칸이 벽이었을 수도 있으니 `map[i]`를 `1`로 바꿔 준다.

`int map_widget::get_visited() / get_cost()` 계산 결과를 읽어오는 함수이다. `get_cost`는 도착점이 없거나 계산 전이면 `-1`을 돌려준다.

`int map_widget::heuristic(int i)` A*에 쓰이는 예상 거리이다. 인덱스를 `x`, `y`로 되돌린 뒤 도착점까지의 맨해튼 거리를 구한다.

```
int x1 = i % w;
```

```
int y1 = i / w;
```

```
int x2 = goal % w;
```

```
int y2 = goal / w;
```

```
return abs(x1 - x2) + abs(y1 - y2); // 맨해튼 거리
```

상하좌우 4방향으로만 움직이기 때문에 맨해튼 거리를 썼다. 대각선 이동이 없으므로 실제 거리보다 커지는 일이 없어 최단 경로가 보장된다.

`void map_widget::paintEvent(QPaintEvent *)` 화면을 그리는 함수이다. Day2의 `refresh_view`와 비슷한 역할이지만, 이쪽은 직접 부르는 게 아니라 `update()`를 호출하면 Qt가 알아서 불러 준다.

칸 하나의 픽셀 크기를 위젯 크기에서 계산한 뒤 이중 `for`문으로 전체 칸을 돌면서 벽이면 검정, 길이면 흰색으로 칠하고 격자선을 그린다. 그 위에 방문한 칸(연보라), 경로(노랑), 시작점(초록), 도착점(빨강) 순서로 덧칠한다. 나중에 칠한 것이 위로 올라오기 때문에 순서가 중요하다.

`void map_widget::mousePressEvent(QMouseEvent *e)` 마우스를 눌렀을 때 실행된다. 눌린 픽셀 좌표를 칸 크기로 나눠서 몇 번째 칸인지 구한다.

```
int x = e->position().x() / cell; // 픽셀 -> 칸 번호
```

```
int y = e->position().y() / cell;
```

맵 밖을 눌렀으면 `return`으로 바로 종료한다. 모드에 따라 벽, 시작점, 도착점을 찍고 그때마다 `twin`에도 똑같은 함수를 불러 두 맵이 항상 같은 상태가 되게 한다. 벽 모드에서 시작점이나 도착점을 누르면 예외처리로 아무것도 하지 않는다.

`void map_widget::run_dijkstra()` 다익스트라 계산을 하는 함수이다.

시작점이나 도착점이 없으면 `return`으로 종료한다. `g`는 전부 `1000000`(사실상 무한대)으로, `parent`는 `-1`, `closed`는 `false`로 초기화하고 시작점의 `g`만 `0`으로 둔다.

우선순위 큐는 `std::priority_queue`에 `std::greater`를 넣어 값이 작은 것이 먼저 나오게 했다. `pair`의 `first`에 비용, `second`에 칸 번호를 넣으면 비용 기준으로 정렬된다.

```
while (!open.empty())
```

```
{
```

```
    int cur = open.top().second;
```

```
    open.pop();
```

```
    if (closed[cur]) { continue; }
```

```
    closed[cur] = true;
```

```
    visited++;
```

```

    if (cur == goal) { break; }
    ...
}

```

큐에서 가장 비용이 작은 칸을 꺼내 확정하고 visited를 하나 올린다. 이미 확정된 칸이면 건너뛰는데, 같은 칸이 큐에 여러 번 들어갈 수 있기 때문이다. 도착점이 나오면 반복을 끝낸다.

그 다음 dx, dy 배열로 상하좌우 네 칸을 확인해서 맵 밖이거나 벽이거나 이미 확정된 칸이면 건너뛰고, 아니면 새 비용을 계산해 기존보다 작을 때만 갱신하고 큐에 넣는다. 한 칸 이동 비용은 1로 두었다.

마지막에 도착점부터 parent를 따라 거꾸로 올라가면서 path에 담고 update()를 불러 화면을 갱신한다.

void map_widget::run_astar() A* 계산을 하는 함수이다. 위의 다익스트라와 구조가 완전히 같고 큐에 넣는 값만 다르다.

```

open.push({ng + heuristic(nxt), nxt}); // f = g + h

```

다익스트라가 $f = g$ 로 지금까지 쓴 비용만 보고 순서를 정하는 반면, A*는 여기에 도착점까지의 예상 거리를 더한 $f = g + h$ 로 순서를 정한다. 그래서 도착점 방향에 있는 칸을 먼저 꺼내게 되고 엉뚱한 방향을 덜 뒤지게 된다.

4. 실행 결과

첨부된 영상 참고

5. 어려웠던 점

가장 헷갈렸던 것은 paintEvent였다. 처음에는 화면을 바꾸고 싶을 때 paintEvent를 직접 부르려고 했는데 그렇게 하면 안 되고 update()를 불러야 한다는 것을 나중에 알았다.

그리고 맵을 1차원 vector로 잡아서 $i = y * w + x$ 로 쓰다 보니 x와 y가 헷갈려 인덱스를 반대로 쓰는 실수를 몇 번 했다. 특히 다시 x, y로 되돌릴 때 $i \% w$ 가 x고 i / w 가 y인데 이걸 바꿔 써서 경로가 이상하게 나온 적이 있었다.

Qt Designer에서 만든 위젯을 커스텀 클래스로 승격시키는 부분도 처음이라 시간이 걸렸다. 승격을 하지 않으면 ui->map_area가 그냥 QWidget이라 우리가 만든 함수를 못 쓴다는 걸 알기까지 헤맸다.